

QuickBite – Smart Retail Food Ordering Platform

Problem Statement

Customers need a seamless way to browse, order, and receive food items like pizza, drinks, and breads without delays or confusion.

Traditional systems lack real-time inventory updates, secure APIs, and smooth user experience.

Objective

To build a scalable, secure, and user-friendly food ordering platform with real-time inventory management.

Key Goals

- Seamless browsing and ordering experience
- Real-time inventory tracking
- Secure authentication & authorization
- Fast and efficient order processing
- Scalable backend architecture

Proposed Solution

A full-stack web application where users can:

- Browse products by categories/brands
- Add items to cart and place orders
- Receive confirmation instantly

The system ensures:

- Inventory auto-update on order confirmation
- Secure API access using JWT
- Transaction-safe order placement

Key Features

- Centralized menu (brands, categories, packaging)
- Product browsing & filtering

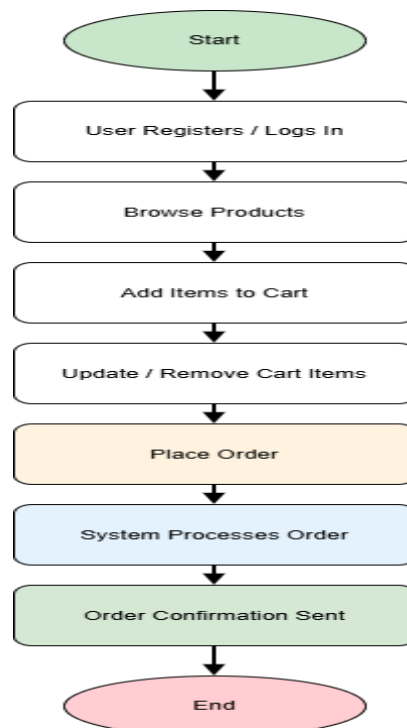
- Cart management
- Order placement with transaction safety
- Automatic inventory deduction
- JWT Authentication & Role-based Authorization
- Admin panel for product & inventory management
- REST APIs tested via Swagger/Postman

Target Users

- Customers (end users placing orders)
- Admins (manage products, inventory, users)

Workflow / User Flow

1. User registers/logs in
2. User browses products
3. Adds items to cart
4. Updates/removes cart items
5. Places order
6. Backend:
 - Creates order
 - Inserts order items
 - Deducts inventory
 - Confirms transaction
7. User receives confirmation



System Architecture

Frontend

- Angular (Standalone Components)
- Handles UI, routing, API calls

Backend

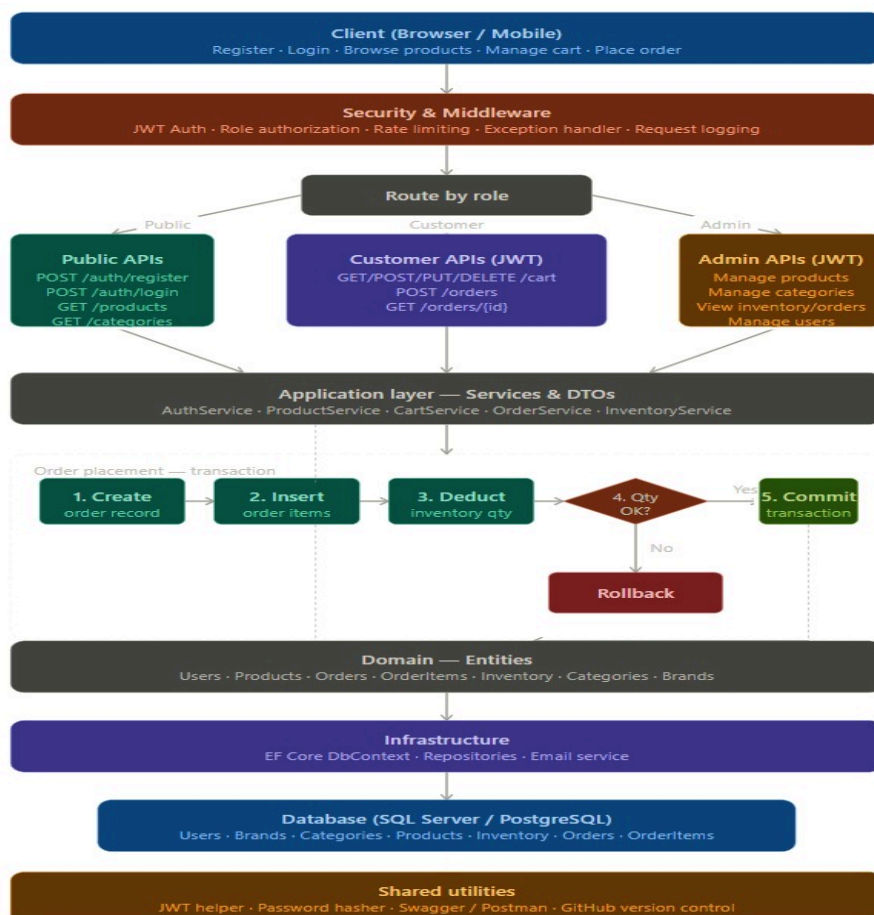
- .NET Web API
- Clean Architecture (Controller → Service → Repository)

Database

- SQL Server
- Entity Framework Core

Integrations

- JWT Authentication
- Swagger / Postman
- Email Service (stretch)



UI/UX Design (Important)

1. Login/Register Screen

- Purpose: Authentication
- Components:
 - Email input
 - Password input
 - Login/Register button

2. Dashboard / Product Listing

- Purpose: Browse products
- Components:
 - Category filter
 - Product cards
 - Add to cart button

3. Cart Screen

- Purpose: Manage cart
- Components:
 - Item list
 - Quantity update
 - Remove button
 - Checkout button

4. Order Confirmation Page

- Purpose: Show order success
- Components:
 - Order ID
 - Summary
 - Status

5. Admin Dashboard

- Purpose: Manage system
- Components:
 - Add product
 - View inventory
 - Manage users/orders

API Endpoints

Public Endpoints (No Auth Required)

Module	Method	Endpoint	Description
Auth	POST	/api/auth/register	Register a new customer
Auth	POST	/api/auth/login	Login and receive JWT token
Products	GET	/api/products	Browse all products
Products	GET	/api/products/{id}	Get product details
Products	GET	/api/products/category/{id}	Filter by category
Categories	GET	/api/categories	List all categories

Customer Endpoints (JWT Required — Role: Customer)

Module	Method	Endpoint	Description
Cart	GET	/api/cart	View cart items
Cart	POST	/api/cart	Add item to cart
Cart	PUT	/api/cart/{itemId}	Update cart item quantity
Cart	DELETE	/api/cart/{itemId}	Remove item from cart
Cart	DELETE	/api/cart/clear	Clear entire cart
Orders	POST	/api/orders	Place a new order
Orders	GET	/api/orders/{id}	View specific order details

Admin Endpoints (JWT Required — Role: Admin)

Module	Method	Endpoint	Description
Products	POST	/api/admin/products	Create a new product
Products	DELETE	/api/admin/products/{id}	Delete a product
Categories	POST	/api/admin/categories	Add a new category
Categories	DELETE	/api/admin/categories/{id}	Delete category
Inventory	GET	/api/admin/inventory	View all inventory levels
Orders	GET	/api/admin/orders	View all customer orders

Users	GET	/api/admin/users	List all registered users
Users	DELETE	/api/admin/users/{id}	Remove a user account

Database Design

Key Tables

Users

- Id (PK)
- Name
- Email
- PasswordHash
- Created At

Products

- Id
- Name
- BrandId
- CategoryId
- Price
- PackagingType

Inventory

- Id
- ProductId
- Quantity

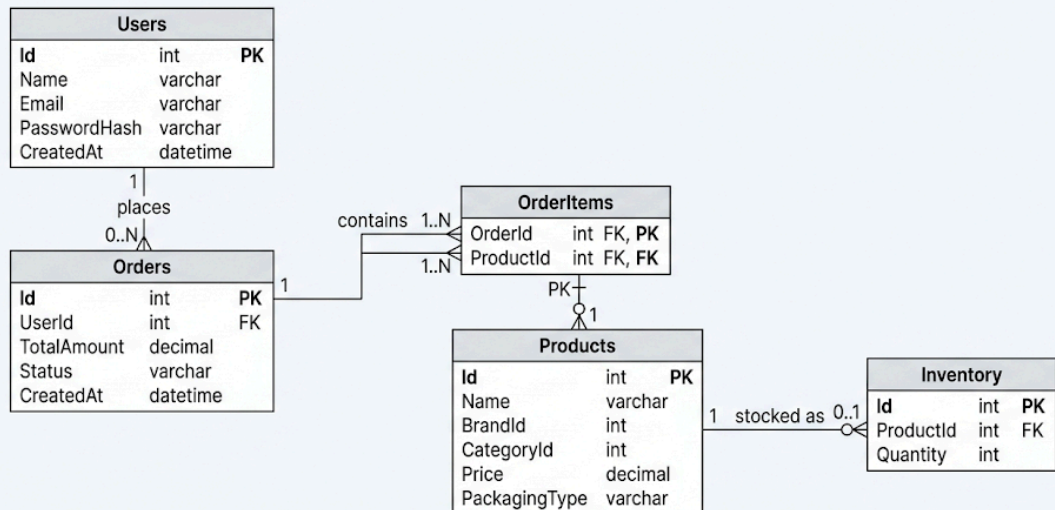
Orders

- Id
- UserId
- TotalAmount
- Status
- Created At

OrderItems

- OrderId
- ProductId
- Quantity
- Price

Database Entity-Relationship Diagram (ER Diagram)



Tech Stack

Frontend

- Angular 17+ (Standalone)

Backend

- .NET Web API
- Entity Framework Core

Database

- SQL Server

Tools/APIs

- Swagger
- GitHub

Scalability & Performance

- Stateless APIs → horizontal scaling

- Database indexing (ProductId, UserId)
 - Caching (future: Redis)
 - Efficient transactions for order handling
-

Security Considerations

- JWT Authentication
 - Role-based Authorization (Admin/Customer)
 - Password hashing
 - Rate limiting
 - Middleware-Global exception handling
-

Future Enhancements

- Email notifications
- Order History
- Loyalty & coupon system